

Reviewer Pack — Young-Flattening Rank Gap Lower-Bounds Padded-Permanent Bord...

Entry 0e80f2d2c50a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 01:05:19 UTC

Young-Flattening Rank Gap Lower-Bounds Padded-Permanent Border Rank

Entry ID: 0e80f2d2c50a **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 01:05:19 UTC

1. Conjecture

Field A (mathematical branch): Geometric Complexity Theory via Young flattenings (Landsberg-Ottaviani equivariant rank obstructions) **Field B** (complexity object): Border rank of the padded $m \times m$ permanent tensor Per_m viewed as a slice of $\det_n$

Statement:

Let Per_m be the $m \times m$ permanent and $\det_n$ its $n \times n$ padded determinant embedding ($n \geq m^2$). Let $\text{YF}_k(T)$ denote the rank of the Young flattening of tensor T indexed by the partition $(k, 1^{\{m-k\}})$ acting on the symmetric m -th power of $C^{\{m\}}$. Conjecture: for all $2 \leq m \leq 6$ and all $1 \leq k \leq m-1$, the equivariant rank gap $\delta(m, k) := \text{YF}_k(\det_n \text{ viewed as } \text{Per}_m\text{-padding}) - \text{YF}_k(\text{Per}_m)$ satisfies $\delta(m, k) \geq c * \binom{m}{k} * (n - m^2 + 1)$ with absolute constant $c = 1/2$, and consequently the border rank lower bound $\text{brk}(\text{Per}_m) \geq \max_k \text{YF}_k(\text{Per}_m) / \binom{m-1}{k-1}$ is tight to within a factor of m .

Rationale (proposer's reasoning):

Young flattenings are the only currently known equivariant obstructions that have produced super-additive border-rank lower bounds for permanents (Landsberg-Michalek), yet no quantitative law connects their rank gaps to the padding parameter n ; pinning down $\delta(m, k)$ would convert orbit-closure containment questions into a finite linear-algebraic check. Because Young flattenings are computed by listing semistandard Young tableaux and writing an explicit polynomial-sized matrix of monomial coefficients, the obstruction is constructive and respects the GL_n^2 -symmetry that defeats natural-proofs and algebrization barriers. A clean quantitative law here would directly sharpen Mulmuley-Sohoni's program for separating VP from VNP via Per vs Det.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fc909a137e0f400a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all (m,k,n) with $m \in \{2,3,4\}$, $1 \leq k \leq m-1$, $n \in \{m^2, \dots, m^2+4\}$, compute $\delta(m,k,n)$ via exact Q-rank Young flattenings (1 deterministic seed; no randomness). SUPPORT iff every cell satisfies $\delta \geq 0.5 \cdot C(m,k) \cdot (n-m^2+1)$ AND $\max_k YF_k(Per_m)/C(m-1,k-1) \in [brk/m, m \cdot brk]$ for $m=2,3$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.97	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.88	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Young flattening permanent determinant border rank Landsberg Ottaviani
- geometric complexity theory padded permanent equivariant rank lower bound - border rank
permanent det_n flattening obstruction binomial partition

Top relevant hits considered: - [s2:10.1145/1993636.1993704] Geometric complexity theory and tensor rank - [s2:1011.1350] Geometric Complexity Theory and Tensor Rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
from fractions import Fraction
from itertools import combinations, permutations

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(n):
            if i != j:
                factor = -A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] += factor * A[i][k]
    return [row[n-1] / row[-2] for row in A]

def young_flattening_rank(tensor, m, k):
    n = len(tensor)
    syts = list(permutations(range(m), k))
    M = [[0] * (m**2) for _ in range(len(syts))]
    for i, syt in enumerate(syts):
        for j in combinations(range(m), m-k):
```

```

        monomial = 1
        for idx in sorted(syt + j):
            monomial *= tensor[idx // m][idx % m]
        M[i][j[0]] = monomial
    return len(gaussian_elimination(M))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for m in [2, 3, 4]:
        C = [[Fraction(1, m**2)] * m**2 for _ in range(m**2)]
        det_n = [[0] * (m**2) for _ in range(m**2)]
        for i in range(m):
            for j in range(m):
                det_n[i][j] = C[i][j]
        YF_k_det_n = young_flattening_rank(det_n, m, 1)
        YF_k_Per_m = young_flattening_rank(C, m, 1)
        delta = YF_k_det_n - YF_k_Per_m
        results.append((m, 1, det_n, C, YF_k_det_n, YF_k_Per_m, delta))
    metric_value = sum(delta for _, _, _, _, _, delta in results) / len(results)
    conjecture_holds = all(delta >= Fraction(1, 2) * binomial(m, k) * (n - m**2 + 1) for m, k, _,
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "delta(m,k)",
        "metric_value": metric_value,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ef5cf2c0.py", line 69, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ef5cf2c0.py", line 50, in run_trial
    YF_k_det_n = young_flattening_rank(det_n, m, 1)
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ef5cf2c0.py", line 39, in young_flatten
    return len(gaussian_elimination(M))
               ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ef5cf2c0.py", line 24, in gaussian_elim
    factor = -A[j][i] / A[i][i]
             ~^^^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` in `gaussian_elimination` before producing any `delta(m,k,n)` data, so the pre-registered support condition could not be evaluated. Per Rule 2, a crash yields INCONCLUSIVE. | next: Fix the `gaussian_elimination` routine (handle zero-pivot/short-row cases, e.g., add partial pivoting and bounds checks) and rerun the deterministic sweep over $m \in \{2,3,4\}$, $1 \leq k \leq m-1$, $n \in \{m^2, \dots, m^2+4\}$.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	28849
2	preregistration	claude_max	opus	0	0	8339
3	novelty	claude_max	opus	0	0	4179
4	novelty	claude_max	opus	0	0	9561
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17524
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9815
7	judge	claude_max	opus	0	0	5579

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 83845 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/0e80f2d2c50a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0e80f2d2c50a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0e80f2d2c50a.tar.gz` (if generated)